

Arrays

JavaScript & TypeScript Essentials

Cloud Contraptions LLC - www.cloudcontraptions.com

What Is an Array?

- A special object that stores an **ordered list** of values
- Common uses: names, scores, todos, cart items
- Values can be mixed types, even functions

```
const empty = [];           // empty
const numbers = [10, 20, 30]; // numbers
const names = ["Sam", "Alex", "Jo"]; // strings
const mixed = [1, "hello", true, { id: 1 }, [2, 3]];
```

Indexes and Length

- Items are numbered from **0** (zero-based)
- `.length` gives the count; last item is `length - 1`

```
const colors = ["red", "green", "blue"];

console.log(colors[0]);    // "red"
colors[1] = "yellow";      // change a value
console.log(colors.length); // 3
console.log(colors[99]);    // undefined - not an error
```

Adding and Removing Items

- End: `push` (add) / `pop` (remove)
- Start: `unshift` (add) / `shift` (remove)

```
const animals = ["cat", "dog"];

animals.push("bird");           // returns the new LENGTH → 3
animals.pop();                  // returns the ITEM removed → "bird"
animals.unshift("mouse");       // ["mouse","cat","dog"]
animals.shift();                 // returns "mouse"
```

- `unshift` / `shift` re-index everything, so they cost more than `push` / `pop`

splice: Insert, Remove, Replace

- `splice(start, deleteCount, ...itemsToInsert)`: all three jobs, in place

```
const items = ["a", "b", "c", "d"];

items.splice(1, 0, "x");    // insert, remove nothing
// ["a", "x", "b", "c", "d"]

items.splice(2, 1);         // remove 1 at index 2 → returns ["b"]
// ["a", "x", "c", "d"]

items.splice(1, 2, "z");    // remove 2, insert 1 → returns ["x", "c"]
// ["a", "z", "d"]
```

- It **returns what it removed** and **mutates** the original

Finding Items

- `includes` → true/false
- `indexOf` → position, or `-1` if not found

```
const letters = ["a", "b", "c", "b"];

letters.includes("b"); // true
letters.includes("z"); // false
letters.indexOf("b");  // 1 (first match)
letters.indexOf("z");  // -1 (not found)
```

- `-1` is truthy, so always compare: `if (arr.indexOf(x) !== -1)`

split and join

- `split` turns a string into an array; `join` turns it back

```
const csv = "red,green,blue";

const colors = csv.split(","); // ["red", "green", "blue"]
colors.join(" | ");           // "red | green | blue"
colors.join("");              // "redgreenblue"

"hello".split("");           // ["h","e","l","l","o"]
```

- They are inverses, and neither changes the original

`slice` Copies, `splice` Mutates

```
const nums = [1, 2, 3, 4, 5];

nums.slice(1, 3); // [2, 3]      - end is EXCLUSIVE
nums.slice(-2);   // [4, 5]      - from the end
nums.slice();     // a full copy
console.log(nums); // [1,2,3,4,5] - untouched

nums.splice(1, 2); // [2, 3] removed
console.log(nums); // [1, 4, 5]  - changed!
```

Looping Over Arrays

- Classic `for` loop uses an index
- `for...of` is simpler when you only need the value

```
const scores = [10, 20, 30];

for (let i = 0; i < scores.length; i++) {
  console.log(scores[i]);
}

for (const score of scores) {
  console.log(score);
}
```

sort: Two Surprises

- It **mutates** the array, and compares as **strings**
- Pass `(a, b) => a - b` for numbers

```
const scores = [10, 9, 100, 1];

scores.sort();           // [1, 10, 100, 9]  string order!
scores.sort((a, b) => a - b); // [1, 9, 10, 100]

// Leave the original alone:
[3, 1, 2].toSorted((a, b) => a - b); // [1, 2, 3]
```

Reading From the End: `at()`

- `at(-1)` beats `arr[arr.length - 1]`
- Negative indexes count back from the end

```
const fruits = ["apple", "banana", "orange"];

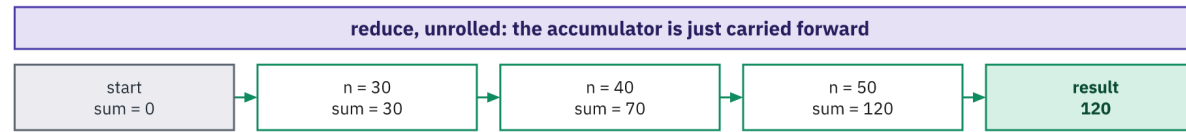
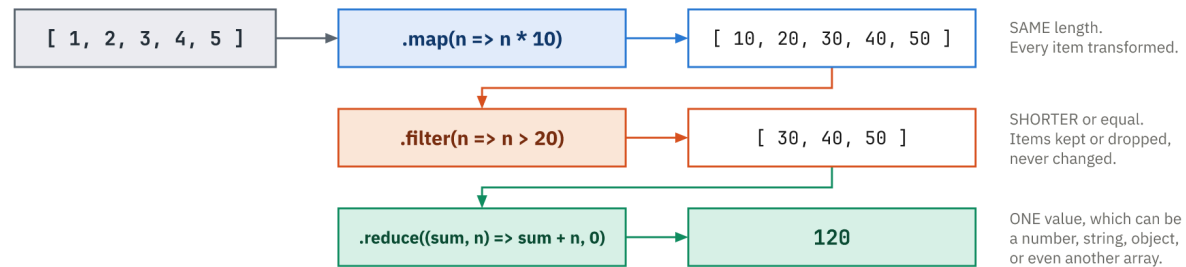
fruits.at(-1); // "orange"
fruits.at(-2); // "banana"
fruits.at(0);  // "apple"

fruits[fruits.length - 1]; // same, but makes you do math
```

map, filter, reduce

map, filter, reduce

Three stages, one data flow. None of them changes the original array; each returns a new value.



Always pass reduce a starting value. Without it, reduce uses the first element as the seed, and throws on an empty array. These return new values, so they chain. `forEach` does not: it returns undefined and exists only for side effects.

- Same length, then shorter, then one value
- None of them changes the original array